

5. Repeat until all vertices are processed.

Shuffle a given array
using Fisher-Yates
Shuffle Algorithm.

Scenario:

Given an array,
generate a random
permutation of array elements

→ "Shuffle a deck of
cards"

→ "Randomize a given
array".

→ Shuffle Means every
permutation of array
element should be equally
likely.

I/P $\rightarrow$

$\{1, 2, 3, 4, 5, 6, 7, 8\}$

O/P $\rightarrow$

$\{7, 8, 1, 2, 3, 4, 6, 5\}$

Brute force Approach

$\rightarrow$ Arr $[]$

$\rightarrow$ Auxiliary Arr $[] \rightarrow$

copy of original array.

$\rightarrow$ Randomly select an element from temp $[]$

$\rightarrow$ copy randomly selected element into Arr $[0]$

$\rightarrow$ Remove selected element from temp $[]$

$\rightarrow$ Repeat the process n times and copy elements

$\rightarrow$ Time complexity $\Rightarrow O(n)^2$

i) Fisher - Yates shuffle

Algorithm:

$\rightarrow$ Function rand $()$ -
Generates a random number in $O(1)$ time.

$\rightarrow$ Key Idea: Start from last element and swap it with randomly selected element from the whole array.

for i from $n-1$ down to 1 do

$J = \text{random integer with } 0 \leq J \leq i$

exchange $a[J]$ and $a[i]$

→ After swapped the selected random value, will be removed.

→ Hence at the end of the loop,

Example $[1^0, 2^1, 3^2]$

$i \quad j$
0 2 (1 index) swap $[2, 1, 3]$ then 2 will be removed from random selecting.
↑ probability = $\left. \begin{array}{l} 0 \rightarrow 3 \\ 1 \rightarrow 2 \\ 2 \rightarrow 1 \end{array} \right\} 6$ permutations totally.

$[1, 2, 3] \Rightarrow 6$ Permutations.

$n=3! = 6$.

$[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 2, 1], [3, 1, 2]$

Time Complexity = $O(n)$

ii) Why Naive Shuffle Algorithm is Biased:

Definition:

Naive shuffle is a simple array-shuffling algorithm in which each element at index i is swapped with a randomly chosen element from the entire array $[0 \text{ to } n-1]$ during every iteration.

Problem:

→ Naive iterative loop generating from

index 0 to $n-1$

→ Utilizing the expression

$\text{swap}[\text{arr}[i], \text{arr}[j]]$

$j = \text{rand}() \% n$

→ It introduces systemic statistical bias.

Mathematical Example:

Array = $[1, 2, 3]$

Naive Shuffle:

```
for (int i = 0; i < n; i++) {
```

```
    j = random
```

```
    int j = rand. nextInt(n);
```

```
    int temp = arr[i];
```

```
    arr[i] = arr[j];
```

```
    arr[j] = temp;
```

```
}
```

Assume:

Random values in the order of $[2, 0, 1]$
[index values]

Loop Table:

Iteration	i	j	swap	Array Before	Array after swapping
1	0	2	$[0, 2]$	$[1, 2, 3]$	$[3, 2, 1]$
2	1	0	$[1, 0]$	$[3, 2, 1]$	$[2, 3, 1]$
3	2	1	$[2, 1]$	$[2, 3, 1]$	$[2, 1, 3]$

Final result: $[2, 1, 3]$

Behind selecting the random value:

Iteration 1 $i=0$ $j = \text{random}(0, 2)$
possible choices $[0, 1, 2]$

count = 3

Iteration 2 $i=1$ $j = \text{random}(0, \overset{\text{Range}}{2})$
possible choices $[0, 1, 2]$

count = 3

Iteration 3 $i=2$ $j = \text{random}(0, 2)$
possible choices $[0, 1, 2]$

count = 3

→ In each Iteration, after selecting the random value, the value doesn't removed. Instead it applied to the upcoming iterations too.

→ Hence total choices will be $\Rightarrow n^n$.
 $3^3 = 27 \text{ choice}$

→ But for permutational random shuffle it will have $n!$

→ (Eg) $3! \Rightarrow 3 \times 2 \times 1 = 6$ permutations.

→ At the final results it will be $n! \neq n^n$.

output permutation counts $\Rightarrow n^n$.

→ This is the reason 'Naive shuffle Algorithm - enumeration paths generated by naive enumeration tree against the actual cardinality.